

category: camera-integration

tags: [ffmpeg, avalonia, kztek-cameras, dead-code, watchdog, race-condition, native-interop]

severity: high

created: 2026-07-20

updated: 2026-07-20

project-origin: parking-v8-app-avalonia (Kztek.Cameras.Avalonia library)

`\_lastFrame` field tồn tại nhưng KHÔNG được gán — mọi API dựa vào nó luôn null/watchdog false-positive

Tình huống gặp phải

Đang sửa `Camera.GetCurrentVideoFrame()` (Kztek.Cameras.Avalonia, `VideoStreamDecoderIntPtr.cs`) để snapshot/capture lấy đúng **độ phân giải camera gốc** (native) thay vì độ phân giải hiển thị theo control — vì `AnvPlayer.GetCurrentVideoFrame()` cũ chỉ đọc từ `_currentAvaloniaFrame` (WriteableBitmap đã resize theo Bounds control).

Thêm method mới `GetCurrentFrameFullResWriteable()` / `GetCurrentFrameSize()` dựa trên field có sẵn `private IntPtr _lastFrame` (đọc dưới `_frameLock`) — nhìn code thấy `GetCurrentFrame(int, int)` cũ cũng đọc từ field này nên tưởng nó đang được cập nhật liên tục bởi decode loop.

Triệu chứng / Lỗi

Sau khi wire `frmViewCamera` (double-click focus window mới) gọi `_player.GetCurrentFrameSize()` mỗi 300ms để hiển thị label — cửa sổ mở lên hiển thị **1 frame tĩnh rồi đứng hình hẳn** sau khoảng 8 giây, dù luồng RTSP/tile gốc vẫn chạy bình thường. Không có exception nào xuất hiện (mọi catch block đều swallow silent).

Nguyên nhân gốc rễ (Root Cause)

`_lastFrame` là field **vestigial (dead)** — chỉ bị RESET về `IntPtr.Zero` khi disconnect (`FFMPEG.av_frame_free(ref _lastFrame)`), nhưng **KHÔNG BAO GIỜ được gán một frame thật** ở bất kỳ đâu trong decode loop hiện tại. `CloneFrame(IntPtr)` — method tồn tại đúng mục đích để clone frame vào `_lastFrame` — được định nghĩa nhưng **0 call site**. Đây là tàn dư từ kiến trúc cũ (trước khi refactor sang Channel-based producer/consumer cho display pipeline); lúc refactor, code populate `_lastFrame` bị rơi mất mà không ai phát hiện vì **KHÔNG** có caller nào active check giá trị trả về là null hay không (mọi API dựa vào field này — `GetCurrentFrame(destWidth, destHeight)` cũ — đơn giản luôn trả `null` và bị bỏ qua âm thầm ở tầng gọi).

Hệ quả với `frmViewCamera.OnTimerTick`: khi `GetCurrentFrameSize()` luôn trả `null` (`_lastFrame` luôn `Zero`), watchdog code coi đây là "8 giây không có frame mới" → tự động gọi `_player.Stop()` — dừng hẳn player, đứng hình vĩnh viễn, dù thực tế luồng vẫn đang nhận frame đều (chỉ là field không phản ánh đúng).

Giải pháp

Populate `_lastFrame` THẬT trong decode loop chính (`TryDecodeFrameLoop`, ngay sau `avcodec_receive_frame` thành công), dùng `CloneFrame()` sẵn có (chỉ `av_frame_ref` — KHÔNG copy pixel, rẻ, an toàn gọi mỗi frame ~30fps):

```
lock (_frameLock)
{
    if (_lastFrame != IntPtr.Zero)
    {
        FFMPEG.av_frame_unref(_lastFrame);
        FFMPEG.av_frame_free(ref _lastFrame);
    }
    _lastFrame = CloneFrame(_pFrame);
}
```

1. Đặt NGAY sau khi `avcodec_receive_frame` trả `r == 0` (frame decode thành công), TRƯỚC hoặc sau

`ConvertFrameToWriteableBitmap(...)` đều được — không phụ thuộc.

2. Free frame CŨ trước khi gán frame MỚI (tránh leak `AVFrame*`).
3. Mọi API đọc `_lastFrame` (`GetCurrentFrame(w,h)`,

`GetCurrentFrameFullResWriteable()`,

`GetCurrentFrameSize()`) PHẢI lock cùng `_frameLock` khi đọc.

Áp dụng lại (How to reuse)

- Khi thấy 1 field/method tồn tại "để phục vụ mục đích X" nhưng method X đó **chưa từng được gọi bởi code hiện tại (**comment kiểu "KHÔNG dùng ở app hiện tại" là dấu hiệu cảnh báo**) → PHẢI grep xác nhận field đó CÓ ĐANG ĐƯỢC GÁN GIÁ TRỊ THẬT** ở đâu đó trong hot path, đừng tin vào tên field hay comment mô tả kiến trúc dự định.
- Trước khi build API mới dựa trên 1 field/private state có sẵn trong native-interop code (FFmpeg, OpenCV, con trỏ thô) → grep TẤT CẢ chỗ gán (`fieldName\s*=\sInterlocked.Exchange, ...`) để xác nhận nó thực sự "live", không phải dead/vestigial.
- Nếu 1 tính năng "đứng hình sau N giây" mà không có exception → nghi ngờ NGAY watchdog/timeout logic dựa trên 1 điều kiện `HasValue/!= null` luôn false do nguồn dữ liệu chưa từng được populate — không phải do mất kết nối thật.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `CloneFrame()` / `av_frame_ref` chỉ tăng refcount + copy metadata, KHÔNG copy pixel — rẻ, an toàn gọi mỗi frame. Đừng nhầm với việc phải convert sang `SKBitmap/WriteableBitmap` mỗi frame (đó mới đắt).

- ⚠ Toàn bộ catch block rỗng (`catch { }`) trong pipeline decode khiến bug loại này KHÔNG BAO GIỜ log ra console/exception — chỉ biểu hiện qua hành vi UI (đứng hình, watchdog dừng player). Khi debug loại freeze không rõ nguyên nhân trong pipeline camera, kiểm tra ngay các watchdog timer trước khi nghi ngờ network/RTSP.
- ⚠ Field `_frameLock` dùng CHUNG cho cả write (decode loop) và read (API snapshot) — nếu thêm API đọc mới, PHẢI lock đúng field này, không tạo lock riêng (race condition đọc `_lastFrame` giữa lúc bị free).

Tham chiếu

- `Kztek.Cameras.Avalonia/Players/FFMPEG/UserControls/VideoStreamDecoderIntpt r.cs` — `_lastFrame`,
`CloneFrame()`, `GetCurrentFrame(int,int)`, `GetCurrentFrameFullResWriteable()`,
`GetCurrentFrameSize()`.
- `Kztek.Cameras.Avalonia/frmViewCamera.axaml.cs` — `OnTimerTick` watchdog (8s timeout → `_player.Stop()`).
- Project liên quan: parking-v8-app-avalonia — `docs/bugs/BUG-004-mainshell-freeze-after-login.md`
(điều tra RAM leak liên quan, cùng pipeline).